

Benchmark Compilation for SAT Competition 2026

Ashlin Iser
Karlsruhe Institute of Technology
Karlsruhe, Germany
ashlin.iser@kit.edu

Marijn Heule
Carnegie Mellon University
Pittsburgh, USA
marijn@cmu.edu

Katalin Fazekas
TU Wien
Vienna, Austria
katalin.fazekas@tuwien.ac.at

Abstract—Compiling a new benchmark set is one of the most impactful tasks in the preparation phase of the SAT Competition. In this document, we report on the instance selection criteria and how they were implemented, as well as new developments in how the benchmark set for the 2026 edition of the SAT Competition was compiled.

Index Terms—benchmark compilation

I. INTRODUCTION

The compilation of benchmarks is an essential part of the SAT Competition. An impartially compiled benchmark set with a controlled bias is the cornerstone of providing a fair assessment of the state of the art in SAT solving. Its impact reaches far beyond the competition itself, as the benchmark set itself is used by researchers to evaluate their solvers and to compare them with the state of the art.

For the SAT Competition 2026, we picked up key developments from previous competitions and developed them further or consolidated them. This document describes the underlying principles and the process of compiling the benchmark set for the SAT Competition 2026 and connects them to the developments in previous competitions.

In Section II, we describe the key developments in previous competitions. Section III describes the preparations that are performed on the submitted instances before the selection process. In Section IV, we describe the selection criteria and how they were implemented in the benchmark compilation script. Section V reports on the statistics of the compiled benchmark set. And finally, Section VI discusses the design decisions critically and provides insights into potential future developments.

II. PREVIOUS DEVELOPMENTS

Previous key developments include the introduction of the “Bring your own benchmarks” (BYOB) rule in SAT Competition 2017 [2], which reinforces the community-driven nature of the SAT Competition by letting participants submit their own instances, enabling them to contribute to what type of instances are considered interesting to solve. What used to be a manual process of hand-picking benchmarks from a large set of submissions has been largely automated in recent years, with the first meta-data driven algorithmic approach in SAT Competition 2020 [3], a competition that also introduced automatic deduplication of identical instances using content-based instance identifiers [6].

Further milestones include the guaranteed exclusion of isomorphic instances, alongside the increasingly meticulous sanitization of submitted instances in SAT Competition 2022 [4]. The first benchmark selection script with reproducible selection results was made publicly available in SAT Competition 2023 [5].

These developments have been picked up and developed further in the benchmark compilation process for SAT Competition 2026. The major change this time is that the benchmark selection script was made publicly available well before the submission deadline on the SAT Competition 2026 website [7], allowing participants and the community to inspect the selection procedure in advance, thereby achieving an unprecedented level of transparency.

III. PREPARATIONS

Each newly submitted benchmark instance is *normalized* such that all comments and superfluous whitespace are removed, and a correct header line is ensured. Instances are then *sanitized* to remove duplicated literals in clauses and tautological clauses which contain both a literal and its negation. Sanitization is performed such that the order of clauses and literals is preserved. In the case of literal deduplication, the first occurrence of the literal is kept, and all subsequent occurrences are removed. If a tautological clause is removed, the header line is updated to reflect the new number of clauses.

In *preanalysis*, we run a set of recent solvers from previous competitions on the newly submitted instances to determine their satisfiability status. This becomes important in the balancing of the benchmark set, as we aim to select a roughly equal number of satisfiable and unsatisfiable instances (cf. Section IV-B). Some of the harder instances may retain an unknown satisfiability status, which is not a problem for the benchmark selection process and is in fact desired to ensure that the benchmark set contains a sufficient number of challenging instances.

We also determine whether instances can be solved by Minisat [1] within a one-minute time limit, which is used to filter out instances that are too easy (cf. Section IV-A). All instance metadata, such as labels indicating the submitting author, or the instance family, their satisfiability status, and their Minisat solvability, is stored in a CSV file that is used as input for the benchmark selection script.

IV. SELECTION CRITERIA

In this section, we revisit the criteria applied in the benchmark compilation process of the 2026 edition of the SAT Competition, describe how they were implemented in the benchmark selection script, and, where applicable, how they were improved compared to previous editions of the competition.

A. Novelty and Hardness

Our selection process first selects among newly submitted benchmark instances, which are contributed either by researchers responding to our *call for benchmarks* or by participants in the competition following the *BYOB rule*. Such newly submitted, previously unseen instances are important to ensure that solvers do not overfit to a small set of known instances, and that the benchmark set contains a sufficient number of challenging instances.

To control hardness, at least ten of these instances must be solvable by the participant's own solver within the competition's time limit, and at least ten of these instances must *not* be solvable by Minisat [1] within a one-minute time limit. We filter out instances that can be solved within a minute by Minisat to ensure that the benchmark set contains a sufficient number of new challenging instances.

The selection of a set of previously known instances in the Global Benchmark Database (GBD) [6] is done by weighted random sampling (Alg. 1, Line 10f.). Before sampling, we restrict the pool of eligible old instances by three filters. First, as for the new submissions, we exclude instances that Minisat can solve within *one minute*, as these are typically too easy to be useful in the benchmark set. Second, we exclude a small number of instances flagged as *huge*: these are enormous but typically easy, with running time dominated by parsing rather than solving, which makes them poor discriminators. Third, we exclude *randomly generated* instances, favoring crafted and application instances that better reflect problems of practical or scientific interest.

Typically, newer instances are more challenging than older instances. To favor novelty, each old instance is assigned a novelty weight W_{novelty} that depends on the first competition year Y in which the instance was used. Instances that were only ever submitted but never actually used in a competition are treated as freshly introduced ($Y = 2025$), whereas instances without any track information receive the baseline weight defined below. For a first use in the years $2017 \leq Y \leq 2025$, the weight grows linearly with recency,

$$W_{\text{novelty}} = 0.8 \cdot \frac{Y - 2016}{2025 - 2016}.$$

Instances first used before 2017, as well as instances without track information, receive a fixed baseline weight of 0.2.

B. Diversity and Balance

The selection of new instances from the submitted pool is done by stratified random sampling, where we stratify by the author of the instance or by the instance family,

Algorithm 1: Benchmark Compilation

Input : Benchmark metadata $\mathcal{M}$

Output: Selected benchmarks $\mathcal{S}$

```

// Select new instances
1 Seed random generator with constant 2026
2 foreach participating author  $a$  do
3    $\mathcal{S}_a \leftarrow$  random sample of up to 12 instances of  $a$ 
4 foreach family  $f$  of a non-participating author do
5    $\mathcal{S}_f \leftarrow$  random sample of up to 16 instances of  $f$ 
6  $\mathcal{S}_{\text{new}} \leftarrow \bigcup_a \mathcal{S}_a \cup \bigcup_f \mathcal{S}_f$ 

// Compute budget for old instances
7 Compute (sat, unsat, unknown) counts of  $\mathcal{S}_{\text{new}}$ 
8 Derive per-status budgets ( $B_{\text{sat}}, B_{\text{unsat}}, B_{\text{unknown}}$ )

// Select old instances
9 Seed random generator with sha256(sorted( $\mathcal{S}_{\text{new}}$ ))
10 foreach old instance  $i$  do
11    $W_i \leftarrow W_{\text{novelty}}(i) + W_{\text{family}}(i) + W_{\text{author}}(i)$ 
12 foreach status  $r \in \{\text{sat}, \text{unsat}, \text{unknown}\}$  do
13    $\mathcal{S}_r \leftarrow$  random sample of  $B_r$  instances using
       weights  $W$ 
14  $\mathcal{S}_{\text{old}} \leftarrow \mathcal{S}_{\text{sat}} \cup \mathcal{S}_{\text{unsat}} \cup \mathcal{S}_{\text{unknown}}$ 
15 return  $\mathcal{S} \leftarrow \mathcal{S}_{\text{new}} \cup \mathcal{S}_{\text{old}}$ 

```

limiting the instance number per *participating* benchmark author to a maximum of 12 (Alg. 1, Line 2f), and per instance family submitted by a *non-participating* benchmark author to a maximum of 16 (Alg. 1, Line 4f). This is supposed to ensure that the benchmark set contains a diverse set of instances from different authors and different families, and that no single author or family dominates the benchmark set.

For the selection of old instances from the GBD pool, we use weights which are inversely proportional to the number of instances in the same family W_{family} or the same author W_{author} . We use $W_{\text{family}} = 100/N_{\text{family}}$ and $W_{\text{author}} = 100/N_{\text{author}}$, where N is the number of instances of the family or author, respectively. The overall sampling weight of an old instance is the sum of its three component weights, $W = W_{\text{novelty}} + W_{\text{family}} + W_{\text{author}}$, and instances are drawn without replacement with probability proportional to W within each satisfiability class (Alg. 1, Line 12f).

For driving the overall set toward an even SAT/UNSAT balance, We use the satisfiability status determined in the preanalysis of the selected new instances, for calculating a budget limiting the number of satisfiable and unsatisfiable instances to be selected from the old instance pool (Alg. 1, Lines 7f and 12). As we target a total of 400 instances in the benchmark, the total old-instance budget is $B = 400 - N_{\text{new}}$, where N_{new} is the number of newly submitted instances that were selected. Let N_{sat} and N_{unsat} denote the number of satisfiable and unsatisfiable instances among the selected new instances. We distribute B over the three satisfiability classes

so as to counterbalance any imbalance already present in the new instances:

$$\begin{aligned} B_{\text{sat}} &= \left\lfloor \frac{B}{3} \right\rfloor + \left\lfloor \frac{N_{\text{unsat}} - N_{\text{sat}}}{2} \right\rfloor, \\ B_{\text{unsat}} &= \left\lfloor \frac{B}{3} \right\rfloor + \left\lfloor \frac{N_{\text{sat}} - N_{\text{unsat}}}{2} \right\rfloor, \\ B_{\text{unknown}} &= B - B_{\text{sat}} - B_{\text{unsat}}. \end{aligned}$$

C. Randomness and Reproducibility

For reproducibility, we use a deterministic seed for the random number generator. The selection of the new instances was seeded with the constant value 2026, while the selection of old instances was seeded with a hash over the identifiers of the selected new instances.

We made the benchmark selection script publicly available [7] well before the submission deadline. Alongside the script we published the metadata database to ensure reproducible query results, allowing anyone to verify and reproduce the benchmark selection process.

We had to make a minor revision to the script to achieve full determinism, as non-determinism was introduced in the previous version of the script due to two issues. The original version iterated over unsorted unique values, whose order is not guaranteed in Polars. It also used Python’s built-in `hash()` function, which is randomized across interpreter invocations via `PYTHONHASHSEED`. Both issues were resolved with minimal, intent-preserving changes: sorting the unique values and replacing `hash()` with `hashlib.sha256()`.

D. Structural Uniqueness

In order to ensure structural uniqueness of the selected instances, we use the IsoHash2 identifier which is GBD’s new, highly efficient, isomorphism-invariant hashing algorithm for SAT instances, based on Weisfeiler-Leman (WL) label refinement [8]. It is a more precise approximation of isomorphism than the previously used degree-sequence based hash, and guarantees that no two selected instances are structurally isomorphic.

V. SELECTION STATISTICS

For the 2026 edition, 19 benchmark authors submitted a total of 1096 instances. Fifteen of these authors were competition participants submitting under the BYOB rule, while four contributed benchmarks only. After discarding instances that Minisat solves within one minute, 1028 remain, of which 994 are structurally distinct according to IsoHash2 [8].

Table I lists, for each submitter, the problem domain and the number of instances at each stage of the pipeline: total of submitted instances (*Sub.*), instances remaining after the Minisat filter (*MsI*), instances structurally unique after IsoHash2 deduplication (*Iso.*), and the instances finally selected after stratified random sampling (*Sel.*).

Table II reports the a priori known satisfiability balance of the final set, broken down into new and old instances. The satisfiability budgeting described in Section IV drives the set toward an even balance, resulting in equal numbers of 146

TABLE I
PER-AUTHOR FUNNEL FROM SUBMITTED TO SELECTED INSTANCES.

Submitter	Domain	Sub.	MsI	Iso.	Sel.
zhenwei	hard-cep, xor-shifting	620	620	609	12
liang	syndrome-decoding	26	23	23	12
anders	graph-coloring	20	20	20	12
green	linear-equations	20	20	20	12
oertel	chnl, count	20	20	20	12
reeves	graph-coloring	20	20	20	12
schreiber	school-timetabling	20	20	20	12
yalun	multiplier-circuit-miters	20	20	20	12
zheng	exam-scheduling	20	20	20	12
gopalan	van-der-waerden	20	18	18	12
hangjiang	stable-semantics	20	12	12	12
ding	argumentation	20	11	11	11
guo	ecc-equivalence-checking	20	10	10	10
biere	lights-out	20	20	7	7
hieu	cyclic-anti-bandwidth	104	101	101	16
heule	allowable-sequence, boxfolding, ntil	54	54	54	48
xudong	datapath-equivalence-checking	16	16	6	6
shiv	sudoku-php	16	2	2	2
faraja	graph-coloring	20	1	1	1
Total		1096	1028	994	233

TABLE II
A PRIORI KNOWN SAT / UNSAT BALANCE OF THE SELECTED BENCHMARK SET.

	SAT	UNSAT	UNKNOWN	Total
New instances	98	84	51	233
Old instances	48	62	57	167
Total	146	146	108	400

satisfiable and 146 unsatisfiable instances, complemented by 108 instances whose status is a priori unknown.

Figures 1 and 2 show how the selected instances are distributed across the 115 instance families and 106 authors in the final set. For readability, families and authors that contribute less than five instances are aggregated by group size k into the *misc-k* buckets.

VI. DISCUSSION

The 2026 benchmark set comprises 400 instances, evenly split between 146 satisfiable and 146 unsatisfiable instances and complemented by 108 instances of a priori unknown status. Of these, 233 stem from new submissions and 167 from the historical pool in the GBD, so that the set reflects the current interests of the community while preserving comparability with previous competitions. The large fraction of unknown instances is intentional and ensures that the set retains a sufficient number of genuinely challenging instances.

In our view, making the selection script publicly available well before the submission deadline was the most consequential step of this edition. It let participants understand exactly how their submissions would be treated and inspect the weighting and balancing criteria in advance. Together with the switch to the more precise IsoHash2 identifier and the deterministic revision of the script, the selection is now reproducible and auditable end to end.

The selection procedure introduces several deliberate biases. The *hardness* uses Minisat as its reference solver, the SAT/UNSAT *balancing* relies on satisfiability and hardness

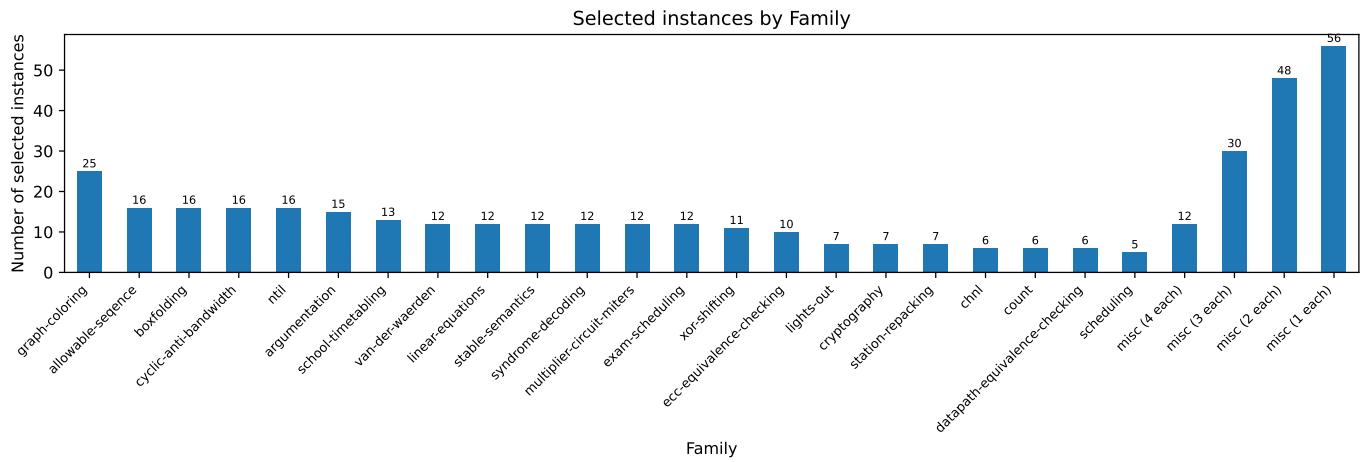


Fig. 1. Number of selected instances per instance family.

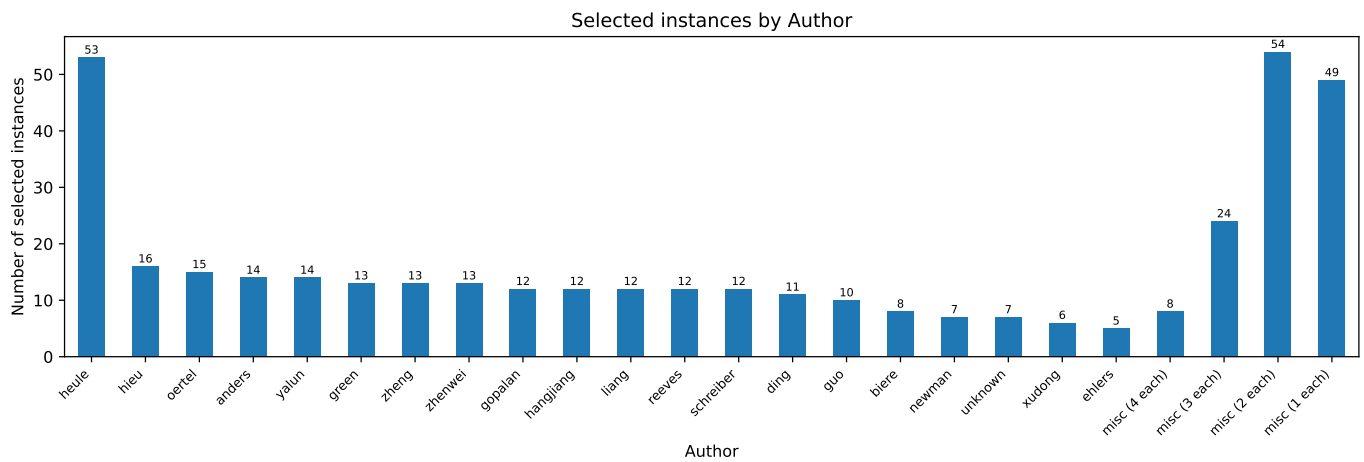


Fig. 2. Number of selected instances per author.

statuses established by solvers from previous competitions, the *novelty* weights favor newer instances, and the *diversity* weights favor less-represented authors and families. Including historical instances may benefit solvers that have been optimized for them. Conversely, the strong emphasis on previously unseen submissions rewards robustness to new challenges and gives the actively submitting community greater influence over the composition of the benchmark set. Excluding randomly generated instances prioritizes crafted and application instances, which are more likely to reflect problems of practical or scientific interest. We consider these biases reasonable and well justified in the context of the competition, while recognizing that they represent design choices rather than uniquely correct decisions and may therefore be revisited in future editions.

REFERENCES

- [1] N. Eén and N. Sörensson, “An Extensible SAT-solver”, in Selected Revised Papers of the Sixth International Conference on Theory and Applications of Satisfiability Testing, 2003, https://doi.org/10.1007/978-3-540-24605-3_37
- [2] Proceedings of SAT Competition 2017: Solver and Benchmark Descriptions, T. Balyo, M. Heule, and M. Järvisalo (editors), <http://hdl.handle.net/10138/231848>
- [3] N. Froleyks, M. Heule, A. Iser, M. Järvisalo, and M. Suda, “The SAT Competition 2020”, Artificial Intelligence, vol. 301, 2021, <https://doi.org/10.1016/j.artint.2021.103572>
- [4] Proceedings of SAT Competition 2022: Solver and Benchmark Descriptions, T. Balyo, M. Heule, A. Iser, M. Järvisalo, and M. Suda (editors), <http://hdl.handle.net/10138/359079>
- [5] A. Iser, “Benchmark Compilation for SAT Competition 2023.”, in Proceedings of SAT Competition 2023: Solver, Benchmark and Proof Checker Descriptions, <http://hdl.handle.net/10138/563824>
- [6] A. Iser, C. Jabs, “Global Benchmark Database”, in Proceedings of SAT 2024, <https://doi.org/10.4230/LIPIcs.SAT.2024.18>
- [7] K. Fazekas, M. Heule, A. Iser, Webpage of SAT Competition 2026, <https://satcompetition.github.io/2026/>
- [8] A. Iser and F. Gehm, “Efficient Identification of Isomorphic SAT Instances”, in Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT), 2026, <https://doi.org/10.4230/LIPIcs.SAT.2026.35>